

C4 Correlation Engine — Visual Structure

Diagram walkthrough of `core/c4/correlation.py` — Stage 3 of the C4 pipeline (WebSentinel R26-CS-003)

This engine takes the flat list of events produced by Stage 1 (extraction) and Stage 2 (rules), and looks for patterns **across different artifact types** — connections a single-event rule could never see on its own. It runs seven independent detectors and returns all their findings together.

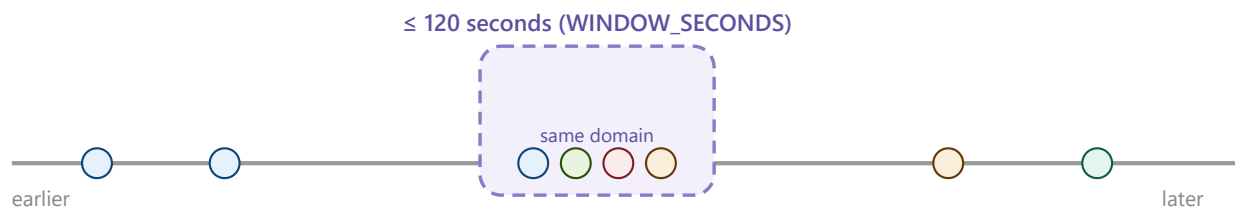
H history **CK** cookie **CR** credential **DL** download **EX** extension **LS** local storage **SS** session/tab

Where correlation sits in the C4 pipeline



`run_correlation(events)` is called once, between the rule engine (Stage 2) and the MITRE mapper (Stage 4)

The core mechanism — a sliding time window



Every event that carries a timestamp becomes an "anchor." All other events within the next 120 seconds are grouped with it, then grouped again **by domain**. Each of the 7 detectors below scans these domain-groups for

The seven detectors — overview

A Co-occurrence

4+ different artifact types cluster on one domain in one window → strong sign that domain is the center of the attack.

B Orphan detection

A cookie / credential / download exists for a domain with **no** matching history visit — injected or history was cleared.

C Temporal anomaly

Activity happens at an hour this specific user is never normally active — personalised baseline, not a fixed clock rule.

D Attack chain

A strict **ordered** sequence — e.g. browse → download → credential — on one domain within the window.

E Domain risk clustering

Every domain gets an overall score from diversity + flagged events + density — ranks the riskiest domain top.

F Credential reuse

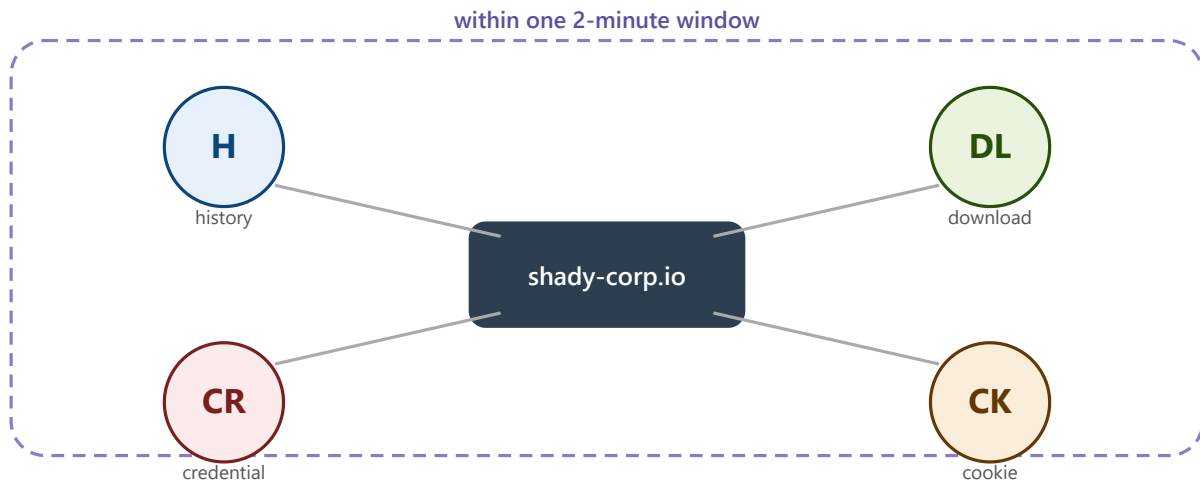
The same saved username appears against 2+ different domains — one stolen password, several accounts exposed.

G Download → exfiltration

A file download followed within 2 minutes by navigation to a *different* domain — drop tool, then call home.

A · Co-occurrence

run_cooccurrence()

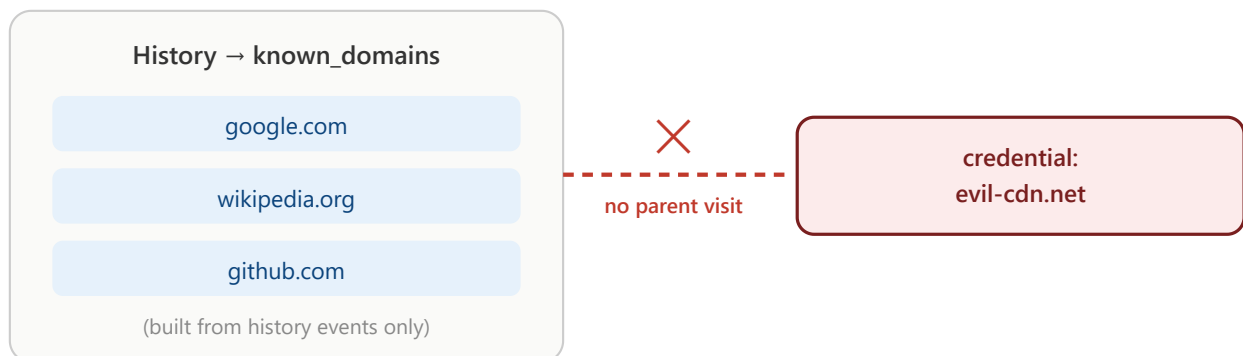


4 artifact types on one domain → COOCCURRENCE_SCORES[4] = 85, boosted further if any are already rule-flagged

Groups events by domain inside each window; a domain touched by 2+ artifact types becomes a finding. More types present → higher score (1→0, 2→40, 3→70, 4→85, 5→100).

B · Orphan detection

run_orphan_detection()

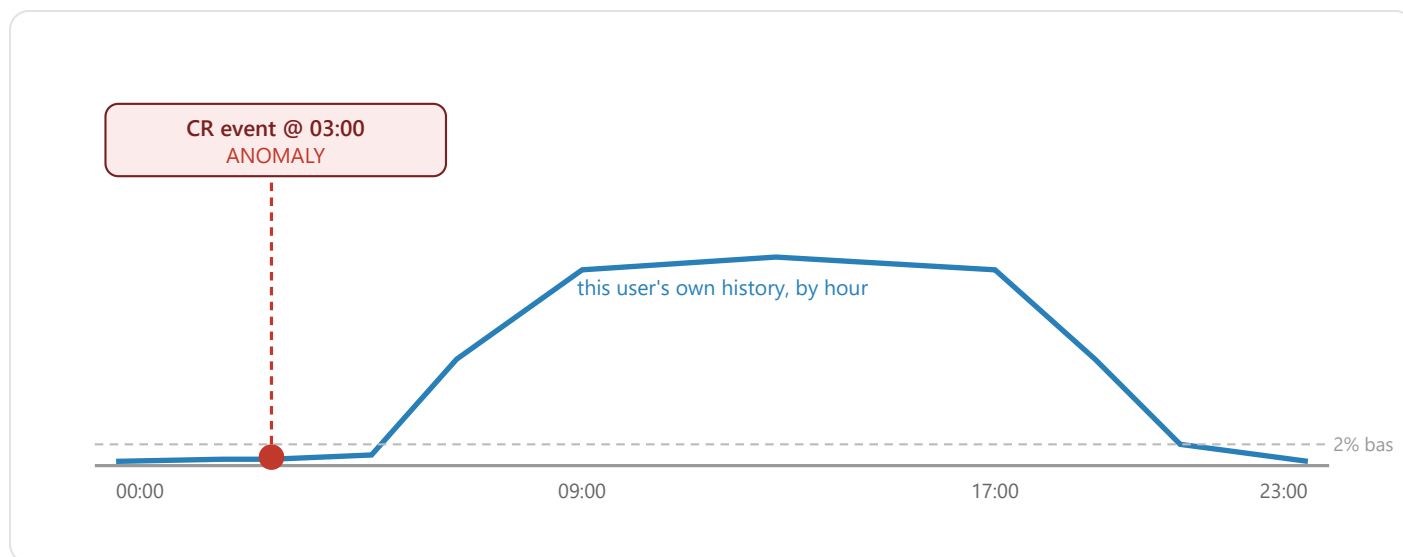


Domain not in known_domains → risk_flag=True, anomaly_score += ORPHAN_SCORES["credential"] (60)

Applies to cookie, credential, download, extension, localstorage and session events alike — an open tab whose URL was never visited (history cleared, or the page never finished loading) is caught the same way.

C · Temporal anomaly

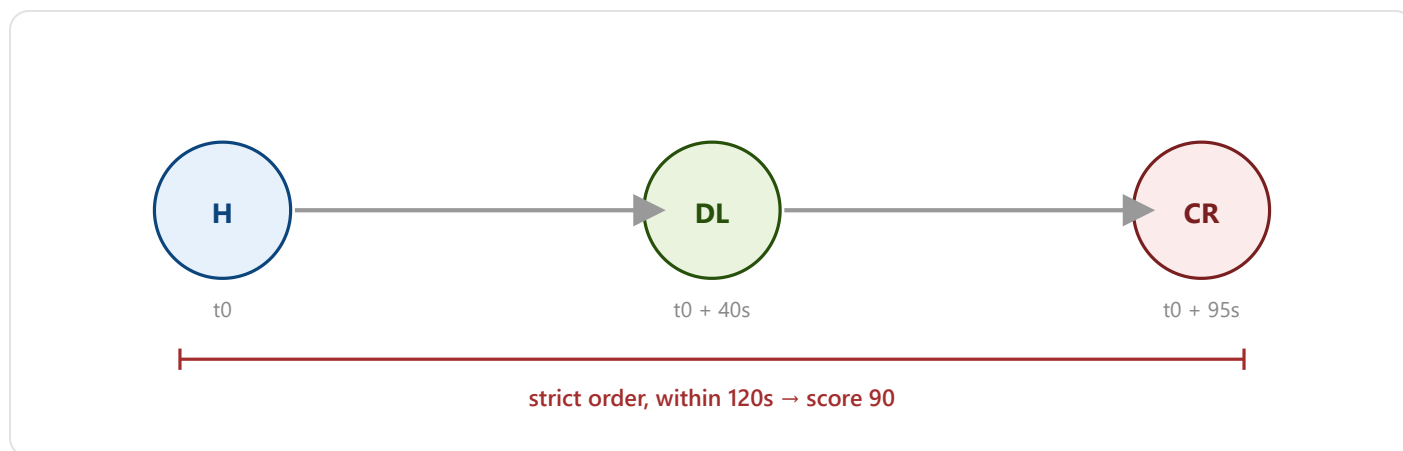
`run_temporal_anomaly()`



Builds an hourly baseline purely from **this user's own** history (needs 10+ history events). Any hour with $\leq 2\%$ of their total activity is "personally inactive"; a risky event landing there is flagged — a threshold personalised per user, not a fixed "2am is suspicious" rule.

D · Attack chain

`run_attack_chain_detection()`

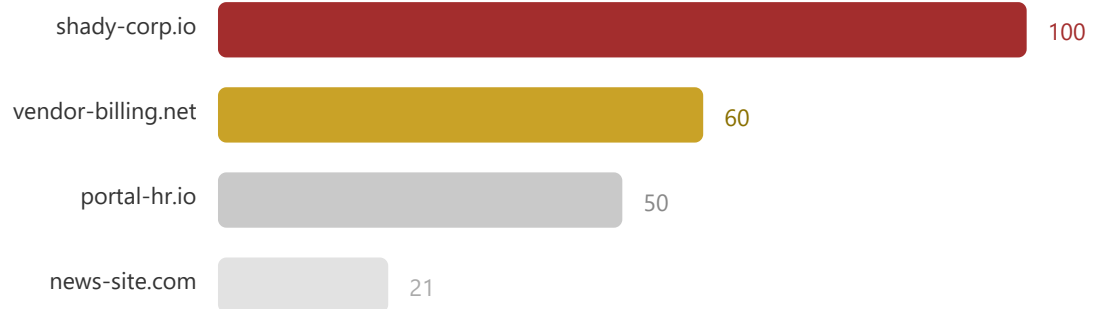


Checks a small set of known dangerous CHAIN_PATTERNS (e.g. history→download→credential = 90, credential→cookie = 70) — stricter than detector A because the sequence must match **in that exact order**, not just co-occur.

E · Domain risk clustering

run_domain_risk_clustering()

score = diversity + flagged_count + density + rule_score, ranked highest first

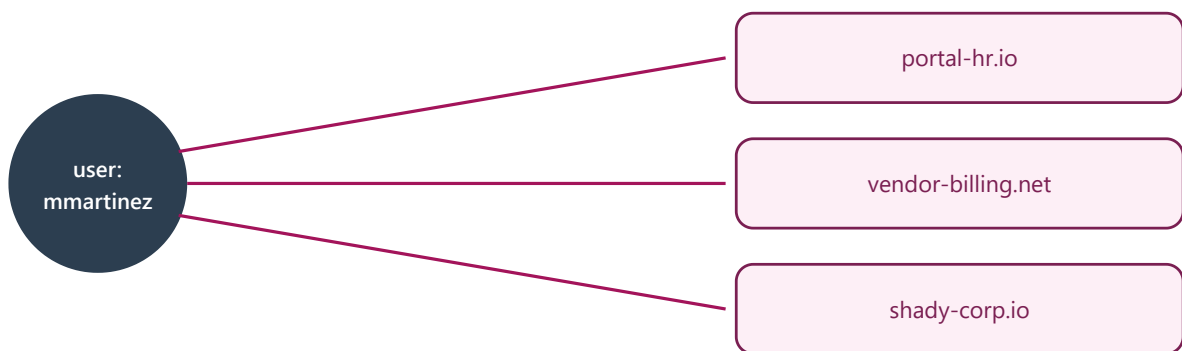


(below 45 → not reported)

Only domains scoring 45+ with 2+ artifact types are reported. The breach domain typically ranks #1 — this is your "which domain deserves attention first" leaderboard.

F · Credential reuse

run_credential_reuse()



3 domains → score $45 + (3-1) \times 20 = 85$ — one leaked password unlocks all three

Groups saved credentials by username; 2+ distinct origin domains for the same username is the finding — a blast-radius signal invisible if you inspect the login table one row at a time.

G · Download → exfiltration

run_download_exfil()



Different domain than the download's own source → drop tool, then call home. Score 45 (or 60 if the download was already rule-flanned)

Bounded to the first follow-on navigation per download to keep noise down — the classic "malware lands, then phones out" step.

What run_correlation() returns

```
run_correlation(events) ->
{
  "cooccurrence": [ ...A findings... ],
  "orphans": [ ...B findings... ],
  "temporal": [ ...C findings... ],
  "attack_chains": [ ...D findings... ],
  "domain_clusters": [ ...E findings... ],
  "credential_reuse": [ ...F findings... ],
  "download_exfil": [ ...G findings... ],
  "baseline": { hour: visit_count, ... }, # from detector C
  "day_hour_baseline": { "weekday_hour": count, ... }, # from detector C
  "summary": {
    "cooccurrence_count": n, "orphan_count": n, "temporal_count": n,
    "attack_chain_count": n, "domain_cluster_count": n,
    "credential_reuse_count": n, "download_exfil_count": n,
    "total_findings": sum of all seven counts
  }
}
```

This whole structure is what service.py passes to mitre.py next (Stage 4) — every finding here gets labeled with a MITRE ATT&CK technique before reaching the reports.

